

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

from sklearn.ensemble import RandomForestRegressor
import joblib
```

```
from google.colab import files
```

```
print('Please upload the CarPrice_Assignment.csv file:')
uploaded = files.upload()
```

Please upload the CarPrice_Assignment.csv file:

[Choose Files](#) 2 files

CarPrice_Assignment_dataset.csv(text/csv) - 26717 bytes, last modified: 6/9/2026 - 100% done

CarPrice_Assignment_dataset.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 14884 bytes, last modified: 6/9/2026 - 100% done

Saving CarPrice_Assignment_dataset.csv to CarPrice_Assignment_dataset (1).csv

Saving CarPrice_Assignment_dataset.xlsx to CarPrice_Assignment_dataset (1).xlsx

After uploading the file, you can now read it into a pandas DataFrame. The `files.upload()` function returns a dictionary where keys are filenames and values are the file contents. We'll get the filename from the keys of this dictionary.

```
import io
csv_file_name = None
for filename in uploaded.keys():
    if filename.endswith('.csv'):
        csv_file_name = filename
        break

if csv_file_name:
    df = pd.read_csv(io.BytesIO(uploaded[csv_file_name]))
    print(f"Successfully loaded {csv_file_name}")
    print(df.head())
else:
    print("No CSV file found in the uploaded files.")
    df = None
```

Successfully loaded CarPrice_Assignment_dataset (1).csv

	car_ID	symboling	CarName	fueltype	aspiration	doornumber	\
0	1	3	alfa-romero giulia	gas	std	two	
1	2	3	alfa-romero stelvio	gas	std	two	
2	3	1	alfa-romero Quadrifoglio	gas	std	two	
3	4	2	audi 100 ls	gas	std	four	
4	5	2	audi 100ls	gas	std	four	

	carbody	drivewheel	engine	location	wheelbase	...	enginesize	\
0	convertible	rwd	front	88.6	...		130	
1	convertible	rwd	front	88.6	...		130	
2	hatchback	rwd	front	94.5	...		152	
3	sedan	fwd	front	99.8	...		109	
4	sedan	4wd	front	99.4	...		136	

	fuelsystem	boreratio	stroke	compressionratio	horsepower	peakrpm	citympg	\
0	mpfi	3.47	2.68	9.0	111	5000	21	
1	mpfi	3.47	2.68	9.0	111	5000	21	
2	mpfi	2.68	3.47	9.0	154	5000	19	
3	mpfi	3.19	3.40	10.0	102	5500	24	
4	mpfi	3.19	3.40	8.0	115	5500	18	

	highwaympg	price
0	27	13495.0
1	27	16500.0
2	26	16500.0
3	30	13950.0
4	22	17450.0

[5 rows x 26 columns]

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205 entries, 0 to 204
Data columns (total 26 columns):
#   Column              Non-Null Count  Dtype
#   ...
```

```

---
0  car_ID          205 non-null    int64
1  symboling       205 non-null    int64
2  CarName         205 non-null    object
3  fueltype        205 non-null    object
4  aspiration       205 non-null    object
5  doornumber       205 non-null    object
6  carbody         205 non-null    object
7  drivewheel       205 non-null    object
8  enginelocation   205 non-null    object
9  wheelbase        205 non-null    float64
10 carlength        205 non-null    float64
11 carwidth         205 non-null    float64
12 carheight        205 non-null    float64
13 curbweight       205 non-null    int64
14 enginetype       205 non-null    object
15 cylindernumber   205 non-null    object
16 enginesize        205 non-null    int64
17 fuelsystem       205 non-null    object
18 boreratio        205 non-null    float64
19 stroke           205 non-null    float64
20 compressionratio 205 non-null    float64
21 horsepower       205 non-null    int64
22 peakrpm          205 non-null    int64
23 citympg          205 non-null    int64
24 highwaympg       205 non-null    int64
25 price            205 non-null    float64
dtypes: float64(8), int64(8), object(10)
memory usage: 41.8+ KB

```

```
df.describe()
```

	car_ID	symboling	wheelbase	carlength	carwidth	carheight	curbweight	enginesize	boreratio	stroke
count	205.000000	205.000000	205.000000	205.000000	205.000000	205.000000	205.000000	205.000000	205.000000	205.000000
mean	103.000000	0.834146	98.756585	174.049268	65.907805	53.724878	2555.565854	126.907317	3.329756	3.255415
std	59.322565	1.245307	6.021776	12.337289	2.145204	2.443522	520.680204	41.642693	0.270844	0.313597
min	1.000000	-2.000000	86.600000	141.100000	60.300000	47.800000	1488.000000	61.000000	2.540000	2.070000
25%	52.000000	0.000000	94.500000	166.300000	64.100000	52.000000	2145.000000	97.000000	3.150000	3.110000
50%	103.000000	1.000000	97.000000	173.200000	65.500000	54.100000	2414.000000	120.000000	3.310000	3.290000
75%	154.000000	2.000000	102.400000	183.100000	66.900000	55.500000	2935.000000	141.000000	3.580000	3.410000
max	205.000000	3.000000	120.900000	208.100000	72.300000	59.800000	4066.000000	326.000000	3.940000	4.170000

```
df.isnull().sum()
```

	0
car_ID	0
symboling	0
CarName	0
fueltype	0
aspiration	0
doornumber	0
carbody	0
drivewheel	0
enginelocation	0
wheelbase	0
carlength	0
carwidth	0
carheight	0
curbweight	0
enginetype	0
cylindernumber	0
enginesize	0
fuelsystem	0
boreratio	0
stroke	0
compressionratio	0
horsepower	0
peakrpm	0
citympg	0
highwaympg	0
price	0

dtype: int64

```
df.drop('car_ID', axis=1, inplace=True)
```

```
le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])
```

```
X = df.drop('price', axis=1)

y = df['price']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)
```

RandomForestRegressor ⓘ ?

RandomForestRegressor(random_state=42)

```
y_pred = model.predict(X_test)
```

```
print(y_pred[:5])
```

```
[36511.91 19355.55 8998.43 12947.37 28022.24]
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("MAE:", mae)
```

```
print("MSE:", mse)
```

```
print("RMSE:", rmse)
```

```
print("R2 Score:", r2)
```

```
MAE: 1307.6731219512199
```

```
MSE: 3522224.1624710755
```

```
RMSE: 1876.7589516160767
```

```
R2 Score: 0.9553832687615398
```

```
importance = model.feature_importances_
```

```
features = X.columns
```

```
plt.figure(figsize=(10,6))
```

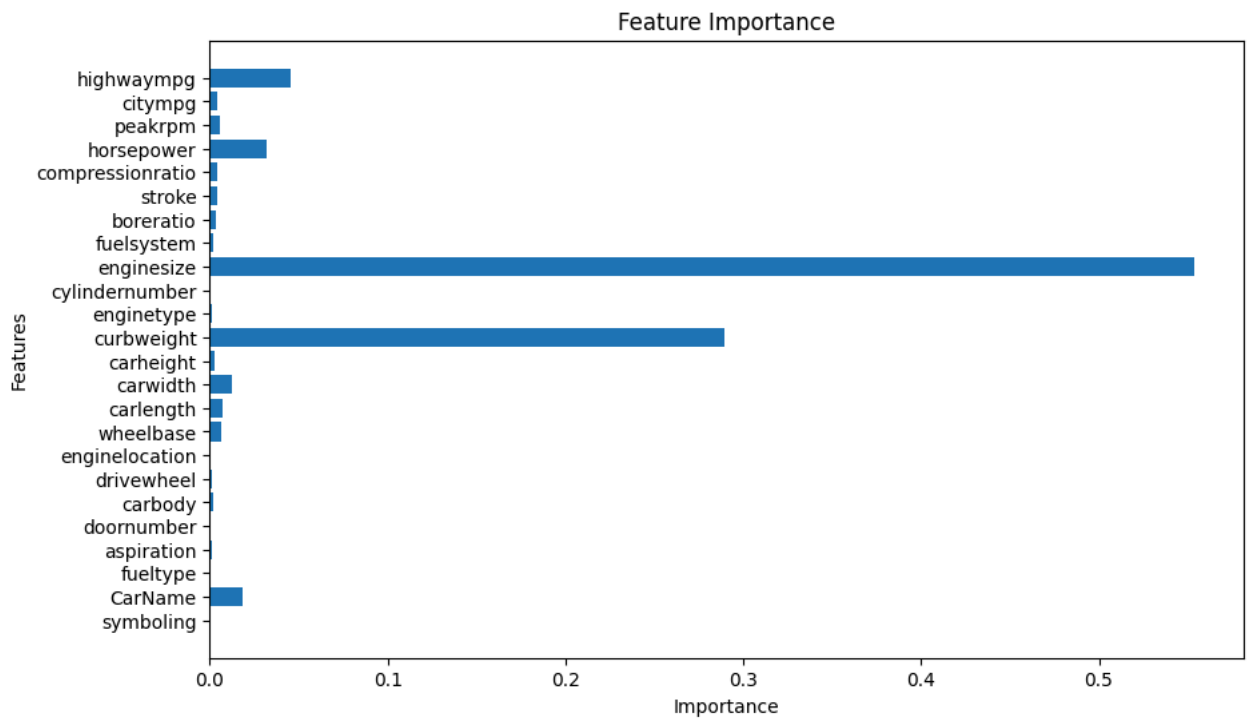
```
plt.barh(features, importance)
```

```
plt.xlabel("Importance")
```

```
plt.ylabel("Features")
```

```
plt.title("Feature Importance")
```

```
plt.show()
```



```
plt.figure(figsize=(8,6))
```

```
plt.scatter(y_test, y_pred)
```

```
plt.xlabel("Actual Price")
```

```
plt.ylabel("Predicted Price")
```

```
plt.title("Actual vs Predicted Car Prices")
```

```
plt.show()
```

